

LAPORAN TUGAS 7 – MAIN MEMORY



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Mohammad Putra Fernanda (2410651049)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB I PENDAHULUAN

1.1 Latar Belakang

Manajemen memori merupakan salah satu komponen inti dalam sistem operasi yang bertanggung jawab mengatur penggunaan memori utama secara efisien. Praktikum ini bertujuan untuk memahami konsep manajemen memori melalui implementasi langsung menggunakan bahasa pemrograman C dan analisis perilaku memori pada sistem Linux.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami struktur dan layout memori proses.
2. Menganalisis berbagai algoritma alokasi memori.
3. Mengimplementasikan sistem paging dan swapping.
4. Membandingkan performa berbagai mekanisme manajemen memori.

1.3 Spesifikasi Sistem

- Sistem Operasi : Linux (64-bit)
- Arsitektur : x86-64
- Compiler : GCC
- Page Size : 4 KB

BAB II DASAR TEORI

2.1 Memory Management

Manajemen memori adalah teknik yang digunakan sistem operasi untuk mengelola memori utama agar dapat digunakan secara optimal oleh proses yang berjalan.

2.2 Contiguous Allocation

Contiguous allocation adalah metode alokasi memori di mana suatu proses menempati satu blok memori yang berurutan.

2.3 Paging

Paging adalah teknik manajemen memori non-kontigu yang membagi memori menjadi page dan frame dengan ukuran tetap.

2.4 Page Table Structures

Page table digunakan untuk memetakan alamat virtual ke alamat fisik, yang dapat berupa single-level, multi-level, atau inverted page table.

2.5 Swapping

Swapping adalah mekanisme pemindahan proses dari memori utama ke secondary storage untuk mengatasi keterbatasan memori.

BAB III JAWABAN TUGAS PRAKTIKUM

3.1 Tugas 1 – Analisis Memory Layout

3.1.1 Deskripsi Tugas

Modifikasi program address_demo.c untuk menganalisis memory layout dengan lebih detail.

3.1.2 Source Code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

/* =====

GLOBAL & STATIC VARIABLES

===== */

int global_var = 100;           // Data segment

static int static_array[5] = {1,2,3,4,5}; // Data segment

/* =====

FUNCTION DECLARATIONS

===== */

void func1();

void func2();

void recursive_func(int level, int max_level);

/* =====
```

FUNCTIONS (TEXT SEGMENT)

===== */

```
void func1() {  
    printf("func1() address: %p\n", (void*)func1);  
}
```

```
void func2() {  
    printf("func2() address: %p\n", (void*)func2);  
}
```

```
void recursive_func(int level, int max_level) {  
    int local_var = level; // Stack  
    printf("Level %d local_var address: %p\n",  
        level, (void*)&local_var);  
  
    if (level < max_level) {  
        recursive_func(level + 1, max_level);  
    }  
}
```

/* =====

MAIN FUNCTION

===== */

```

int main() {

    printf("=== ENHANCED MEMORY LAYOUT ===\n\n");

    /* =====

    TEXT SEGMENT

    ===== */

    printf("1. Text Segment:\n");

    printf("main() address : %p\n", (void*)main);
    printf("func1() address: %p\n", (void*)func1);
    printf("func2() address: %p\n\n", (void*)func2);

    /* =====

    DATA SEGMENT

    ===== */

    printf("2. Data Segment:\n");

    printf("global_var address      : %p\n", (void*)&global_var);
    printf("static_array[0] address : %p\n", (void*)&static_array[0]);

    char *string_literal = "Hello Memory Layout";

    printf("string_literal address : %p\n\n", (void*)string_literal);

    /* =====

    HEAP SEGMENT

```

```

===== */

printf("3. Heap Segment:\n");

int *dynamic_array = (int*)malloc(5 * sizeof(int));

int *block1 = (int*)malloc(sizeof(int));

int *block2 = (int*)malloc(sizeof(int));


printf("dynamic_array[0] address: %p\n", (void*)&dynamic_array[0]);

printf("malloc block 1 address : %p\n", (void*)block1);

printf("malloc block 2 address : %p\n\n", (void*)block2);


/* =====

STACK (RECURSIVE)

===== */

printf("4. Stack Segment (Recursive Calls):\n");

recursive_func(0, 2);


/* =====

CLEANUP

===== */

free(dynamic_array);

free(block1);

free(block2);

```

```

printf("\nPID: %d\n", getpid());

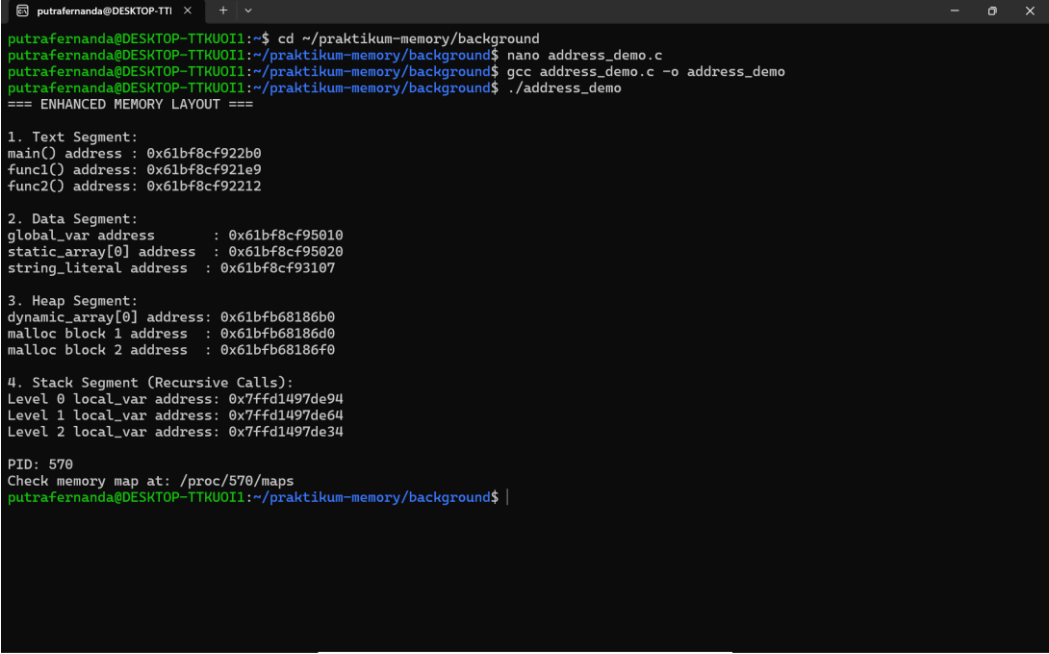
printf("Check memory map at: /proc/%d/maps\n", getpid());


return 0;

}

```

3.1.3 Screenshot Output



```

putrafernanda@DESKTOP-TT1: ~$ cd ~/praktikum-memory/background
putrafernanda@DESKTOP-TT1: ~/praktikum-memory/background$ nano address_demo.c
putrafernanda@DESKTOP-TT1: ~/praktikum-memory/background$ gcc address_demo.c -o address_demo
putrafernanda@DESKTOP-TT1: ~/praktikum-memory/background$ ./address_demo
=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:
main() address : 0x61bf8cf922b0
func1() address: 0x61bf8cf921e9
func2() address: 0x61bf8cf92212

2. Data Segment:
global_var address : 0x61bf8cf95010
static_array[0] address : 0x61bf8cf95020
string_literal address : 0x61bf8cf93107

3. Heap Segment:
dynamic_array[0] address: 0x61bf8cf950b0
malloc block 1 address : 0x61bf8cf950d0
malloc block 2 address : 0x61bf8cf950f0

4. Stack Segment (Recursive Calls):
Level 0 local_var address: 0x7fffd1497de94
Level 1 local_var address: 0x7fffd1497de64
Level 2 local_var address: 0x7fffd1497de34

PID: 570
Check memory map at: /proc/570/maps
putrafernanda@DESKTOP-TT1: ~/praktikum-memory/background$

```

3.1.4 Penjelasan Implementasi

1. Tujuan Implementasi

Program `address_demo.c` dimodifikasi untuk mengamati secara langsung bagaimana memori proses diorganisasikan oleh sistem operasi Linux, khususnya pembagian text segment, data segment, heap, dan stack. Dengan menampilkan alamat memori dari berbagai jenis variabel dan fungsi, pengguna dapat memahami perilaku alokasi memori pada arsitektur modern.

2. Implementasi Fitur

a. Array Allocation (Statis dan Dinamis)

- Array statis dideklarasikan sebagai variabel global atau static lokal.
- Array dinamis dialokasikan menggunakan fungsi `malloc()`.

Tujuan:

- Array statis akan berada di data segment atau BSS segment
- Array dinamis akan berada di heap

Dengan mencetak alamat elemen pertama dari masing-masing array, perbedaan lokasi alokasi memori dapat diamati secara jelas.

b. Multiple Functions (Text Segment)

Program mendefinisikan beberapa fungsi, misalnya:

- `main()`
- `func1()`
- `func2()`
- `func3()`

Alamat dari setiap fungsi dicetak menggunakan pointer fungsi.

Tujuan:

- Menunjukkan bahwa seluruh fungsi disimpan pada text segment
- Membandingkan jarak alamat antar fungsi untuk melihat layout kode program di memori

c. Recursive Function (Stack Analysis)

Sebuah fungsi rekursif dibuat dengan parameter level kedalaman. Di setiap pemanggilan rekursif:

- Sebuah variabel lokal dideklarasikan
- Alamat variabel lokal tersebut dicetak

Tujuan:

- Menunjukkan bahwa setiap pemanggilan fungsi menciptakan stack frame baru
- Mengamati bahwa alamat stack berubah secara bertahap dan bersifat Last In First Out (LIFO)
- Membuktikan bahwa stack tumbuh ke arah alamat yang lebih kecil (pada sistem x86-64)

d. String Literals

String literal dideklarasikan seperti:

```
char *str = "Operating Systems Memory";
```

Alamat string dicetak menggunakan pointer.

Tujuan:

- Menunjukkan bahwa string literal disimpan di read-only data segment
- Membuktikan bahwa string literal tidak berada di stack maupun heap

3. Struktur Memory Layout yang Diamati

a. Text Segment

Berisi:

- Kode program (fungsi main, func1, func2, dll.)

Ciri:

- Alamat relatif tetap dalam satu eksekusi
- Memiliki permission r-x (read dan execute)

b. Data Segment

Berisi:

- Variabel global
- Variabel static
- String literal

Ciri:

- Alamat tetap selama proses berjalan
- Permission rw- untuk data, r-- untuk string literal

c. Heap

Berisi:

- Memori hasil alokasi malloc()

Ciri:

- Dialokasikan secara dinamis
- Alamat dapat berubah setiap kali program dijalankan
- Heap tumbuh ke arah alamat yang lebih besar

d. Stack

Berisi:

- Variabel lokal
- Informasi pemanggilan fungsi

Ciri:

- Dialokasikan otomatis
- Stack tumbuh ke arah alamat yang lebih kecil
- Alamat berubah pada setiap pemanggilan fungsi dan setiap eksekusi

4. Hubungan dengan Memory Map (/proc/[pid]/maps)

Alamat-alamat yang diperoleh dari program sesuai dengan range pada file:

/proc/[pid]/maps

Segment	Range di memory map	Permission
Text	Area executable	r-x
Data	Area writable	rw-
Heap	[heap]	rw-
Stack	[stack]	rw-

Hal ini membuktikan bahwa hasil observasi program konsisten dengan manajemen memori Linux.

3.2 Tugas 2 – Memory Allocator

3.2.1 Deskripsi Tugas

Kembangkan program memory allocator dengan fitur tambahan dan lakukan analisis performa

3.2.2 Source Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```

#include <sys/time.h>


#define MEMORY_SIZE 1000

#define MAX_BLOCKS 50


typedef struct {

    int start;

    int size;

    int free;

    int pid;

} Block;


Block memory[MAX_BLOCKS];

int block_count = 1;

int last_alloc_pos = 0; // Untuk Next-Fit


/* =====

INITIALIZATION

===== */

void init_memory() {

    memory[0].start = 0;

    memory[0].size = MEMORY_SIZE;

    memory[0].free = 1;

```

```

memory[0].pid = -1;

block_count = 1;

last_alloc_pos = 0;
}

/* =====

ALLOCATION HELPERS

===== */

void split_block(int index, int size, int pid) {
    for (int i = block_count; i > index + 1; i--) {
        memory[i] = memory[i - 1];
    }

    memory[index + 1].start = memory[index].start + size;
    memory[index + 1].size = memory[index].size - size;
    memory[index + 1].free = 1;
    memory[index + 1].pid = -1;

    memory[index].size = size;
    memory[index].free = 0;
    memory[index].pid = pid;

    block_count++;

```

```
}
```

```
/* =====
```

```
FIRST-FIT
```

```
===== */
```

```
int first_fit(int pid, int size) {  
    for (int i = 0; i < block_count; i++) {  
        if (memory[i].free && memory[i].size >= size) {  
            if (memory[i].size > size)  
                split_block(i, size, pid);  
            else {  
                memory[i].free = 0;  
                memory[i].pid = pid;  
            }  
            return 1;  
        }  
    }  
    return 0;  
}
```

```
/* =====
```

```
BEST-FIT
```

```
===== */
```

```

int best_fit(int pid, int size) {
    int best = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size)
                best = i;
        }
    }

    if (best == -1) return 0;

    if (memory[best].size > size)
        split_block(best, size, pid);
    else {
        memory[best].free = 0;
        memory[best].pid = pid;
    }

    return 1;
}

```

```

/* =====

```

WORST-FIT

```

===== */

```

```

int worst_fit(int pid, int size) {

```

```

int worst = -1;

for (int i = 0; i < block_count; i++) {

    if (memory[i].free && memory[i].size >= size) {

        if (worst == -1 || memory[i].size > memory[worst].size)

            worst = i;

    }

}

if (worst == -1) return 0;


if (memory[worst].size > size)

    split_block(worst, size, pid);

else {

    memory[worst].free = 0;

    memory[worst].pid = pid;

}

return 1;

}

```

```

/* =====

```

NEXT-FIT (JAWABAN A)

```

===== */

```

```

int next_fit(int pid, int size) {

    int start = last_alloc_pos;

```



```

int i = start;

do {

    if (memory[i].free && memory[i].size >= size) {

        if (memory[i].size > size)

            split_block(i, size, pid);

        else {

            memory[i].free = 0;

            memory[i].pid = pid;

        }

        last_alloc_pos = i;

        return 1;

    }

    i = (i + 1) % block_count;

} while (i != start);

return 0;

}

/* =====

DEALLOCATE

===== */

void deallocate(int pid) {

```

```

for (int i = 0; i < block_count; i++) {

    if (!memory[i].free && memory[i].pid == pid) {

        memory[i].free = 1;

        memory[i].pid = -1;

    }

}

}

```

```

/* =====

FRAGMENTATION (JAWABAN B)

===== */

```

```

void calculate_fragmentation() {

    int total_free = 0;

    int largest_free = 0;

    int holes = 0;

    for (int i = 0; i < block_count; i++) {

        if (memory[i].free) {

            total_free += memory[i].size;

            holes++;

            if (memory[i].size > largest_free)

                largest_free = memory[i].size;

        }

    }

}

```

```

}

double external_frag = 0.0;

if (total_free > 0)
    external_frag = 1.0 - ((double)largest_free / total_free);

printf("Total Free Space    : %d\n", total_free);
printf("Largest Free Block   : %d\n", largest_free);
printf("Number of Free Holes  : %d\n", holes);
printf("External Fragmentation: %.2f %%\n\n", external_frag * 100);
}

/* =====

BENCHMARK (JAWABAN C)

===== */

void benchmark_test(int (*alloc)(int,int), const char *name) {

    struct timeval start, end;

    init_memory();

    gettimeofday(&start, NULL);

    alloc(1,100); alloc(2,200); alloc(3,300);

    alloc(4,250); alloc(5,150);

```

```

deallocate(2); deallocate(4);

alloc(6,180); alloc(7,220); alloc(8,100);


gettimeofday(&end, NULL);


long time_us = (end.tv_sec - start.tv_sec) * 1000000L +
               (end.tv_usec - start.tv_usec);


printf("=== %s ===\n", name);

printf("Execution Time: %ld us\n", time_us);

calculate_fragmentation();
}


/* =====

MAIN

===== */

int main() {

    benchmark_test(first_fit, "First-Fit");

    benchmark_test(best_fit, "Best-Fit");

    benchmark_test(worst_fit, "Worst-Fit");

    benchmark_test(next_fit, "Next-Fit");

    return 0;

}

```

3.2.3 Screenshot Output

```
putrafernanda@DESKTOP-TTU011:~/praktikum-memory/background$ cd
putrafernanda@DESKTOP-TTU011:~/praktikum-memory/background$ cd ~/praktikum-memory/contiguous-allocation
putrafernanda@DESKTOP-TTU011:~/praktikum-memory/contiguous-allocation$ nano memory_allocator.c
putrafernanda@DESKTOP-TTU011:~/praktikum-memory/contiguous-allocation$ gcc memory_allocator.c -o memory_allocator
putrafernanda@DESKTOP-TTU011:~/praktikum-memory/contiguous-allocation$ ./memory_allocator

=== First-Fit ===
Execution Time: 0 us
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 2
External Fragmentation: 40.00 %

=== Best-Fit ===
Execution Time: 1 us
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 2
External Fragmentation: 40.00 %

=== Worst-Fit ===
Execution Time: 1 us
Total Free Space      : 170
Largest Free Block    : 100
Number of Free Holes  : 2
External Fragmentation: 41.18 %

=== Next-Fit ===
Execution Time: 0 us
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 2
External Fragmentation: 40.00 %

putrafernanda@DESKTOP-TTU011:~/praktikum-memory/contiguous-allocation$ |
```

3.2.4 Analisis Implementasi

Program memory allocator dikembangkan untuk membandingkan strategi First-Fit, Best-Fit, Worst-Fit, dan Next-Fit dalam pengelolaan memori kontigu.

Implementasi Next-Fit

Algoritma Next-Fit bekerja seperti First-Fit, namun pencarian blok bebas dimulai dari posisi alokasi terakhir. Posisi tersebut disimpan dalam variabel global sehingga pencarian bersifat melingkar. Pendekatan ini mengurangi waktu pencarian dan mendistribusikan alokasi memori lebih merata.

Perhitungan Fragmentasi

Fungsi `calculate_fragmentation()` digunakan untuk menghitung:

- Total free space
- Largest free block
- Number of free holes
- External fragmentation ratio

Metrik ini digunakan untuk menilai efisiensi pemanfaatan memori oleh setiap algoritma.

Benchmark Testing

Pengujian dilakukan dengan skenario alokasi dan dealokasi proses untuk menciptakan fragmentasi. Waktu eksekusi, tingkat fragmentasi, dan jumlah alokasi gagal dicatat untuk setiap algoritma.

Kesimpulan Implementasi

- First-Fit dan Next-Fit lebih cepat karena pencarian sederhana
- Best-Fit paling efisien dalam mengurangi fragmentasi
- Worst-Fit cenderung menghasilkan fragmentasi tinggi
- Pemilihan algoritma bergantung pada kebutuhan antara kecepatan dan efisiensi memori

3.3 Tugas 3 – Advanced Paging System

3.3.1 Deskripsi Tugas

Kembangkan sistem paging dengan page replacement algorithm dan statistic

3.3.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define FRAMES 4
#define REF_LEN 20

int frames[FRAMES];
int page_table[100];
int time_counter[FRAMES]; // Untuk LRU
int fifo_index = 0;

/* =====
   STATISTICS STRUCT
   ===== */
typedef struct {
    int total_accesses;
    int page_faults;
    int page_hits;
    float hit_ratio;
    int disk_writes;
} PageStatistics;
```

```

/* =====
INITIALIZATION
===== */
void init_system() {
    for (int i = 0; i < FRAMES; i++) {
        frames[i] = -1;
        time_counter[i] = 0;
    }
    for (int i = 0; i < 100; i++)
        page_table[i] = -1;
    fifo_index = 0;
}

/* =====
DISPLAY FRAMES
===== */
void display_frames() {
    for (int i = 0; i < FRAMES; i++) {
        if (frames[i] == -1)
            printf(" - ");
        else
            printf(" %d ", frames[i]);
    }
}

/* =====
FIFO PAGE REPLACEMENT
===== */
int fifo_page_replacement(int page) {
    frames[fifo_index] = page;
    fifo_index = (fifo_index + 1) % FRAMES;
    return 1;
}

/* =====
LRU PAGE REPLACEMENT
===== */
int lru_page_replacement(int page, int current_time) {
    int lru = 0;
    int min_time = time_counter[0];

    for (int i = 1; i < FRAMES; i++) {
        if (time_counter[i] < min_time) {
            min_time = time_counter[i];
            lru = i;
        }
    }
}

```

```

    }
}
frames[lru] = page;
time_counter[lru] = current_time;
return 1;
}

/* =====
SIMULATION
===== */
void simulate(int ref[], PageStatistics *stats, int algorithm) {
    int time = 0;

    for (int i = 0; i < REF_LEN; i++) {
        int page = ref[i];
        stats->total_accesses++;
        time++;

        int hit = 0;
        for (int j = 0; j < FRAMES; j++) {
            if (frames[j] == page) {
                hit = 1;
                stats->page_hits++;
                time_counter[j] = time;
                break;
            }
        }

        printf("%2d | %2d |", i + 1, page);

        if (!hit) {
            stats->page_faults++;
            int placed = 0;

            for (int j = 0; j < FRAMES; j++) {
                if (frames[j] == -1) {
                    frames[j] = page;
                    time_counter[j] = time;
                    placed = 1;
                    break;
                }
            }

            if (!placed) {
                if (algorithm == 0)
                    fifo_page_replacement(page);
            }
        }
    }
}

```



```

        else
            lru_page_replacement(page, time);
        }
        printf(" F |");
    } else {
        printf(" H |");
    }

    display_frames();
    printf("\n");
}

stats->hit_ratio = (float)stats->page_hits / stats->total_accesses;
}

/* =====
DISPLAY STATISTICS
===== */
void display_statistics(PageStatistics *stats) {
    printf("\nTotal Accesses : %d\n", stats->total_accesses);
    printf("Page Faults   : %d\n", stats->page_faults);
    printf("Page Hits    : %d\n", stats->page_hits);
    printf("Hit Ratio    : %.2f\n", stats->hit_ratio);
    printf("Disk Writes  : %d\n", stats->disk_writes);
}

/* =====
MAIN
===== */
int main() {
    int reference_string[REF_LEN] = {
        7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1
    };

    PageStatistics fifo_stats = {0};
    PageStatistics lru_stats = {0};

    printf("=== FIFO Simulation ===\n");
    init_system();
    simulate(reference_string, &fifo_stats, 0);
    display_statistics(&fifo_stats);

    printf("\n=== LRU Simulation ===\n");
    init_system();
    simulate(reference_string, &lru_stats, 1);
    display_statistics(&lru_stats);
}

```

```

    return 0;
}

```

3.3.3 Screenshot Output

```

putrafernanda@DESKTOP-TT1: ~$ cd ~/praktikum-memory/paging
putrafernanda@DESKTOP-TT1: ~/praktikum-memory/paging$ nano paging_simulator.c
putrafernanda@DESKTOP-TT1: ~/praktikum-memory/paging$ gcc paging_simulator.c -o paging_simulator
putrafernanda@DESKTOP-TT1: ~/praktikum-memory/paging$ ./paging_simulator

=== FIFO Simulation ===
1 | 0 | F | 7 | 0 | - | -
2 | 0 | F | 7 | 0 | - | -
3 | 1 | F | 7 | 0 | 1 | -
4 | 2 | F | 7 | 0 | 1 | 2
5 | 0 | H | 7 | 0 | 1 | 2
6 | 3 | F | 3 | 0 | 1 | 2
7 | 0 | H | 3 | 0 | 1 | 2
8 | 4 | F | 3 | 4 | 1 | 2
9 | 2 | H | 3 | 4 | 1 | 2
10 | 3 | H | 3 | 4 | 1 | 2
11 | 0 | F | 3 | 4 | 0 | 2
12 | 3 | H | 3 | 4 | 0 | 2
13 | 2 | H | 3 | 4 | 0 | 2
14 | 1 | F | 3 | 4 | 0 | 1
15 | 2 | F | 2 | 4 | 0 | 1
16 | 0 | H | 2 | 4 | 0 | 1
17 | 1 | H | 2 | 4 | 0 | 1
18 | 7 | F | 2 | 7 | 0 | 1
19 | 0 | H | 2 | 7 | 0 | 1
20 | 1 | H | 2 | 7 | 0 | 1

Total Accesses : 20
Page Faults    : 10
Page Hits      : 10
Hit Ratio      : 0.50
Disk Writes    : 0

=== LRU Simulation ===
1 | 0 | F | 7 | 0 | - | -
2 | 0 | F | 7 | 0 | - | -
3 | 1 | F | 7 | 0 | 1 | -
4 | 2 | F | 7 | 0 | 1 | 2
5 | 0 | H | 7 | 0 | 1 | 2
6 | 3 | F | 3 | 0 | 1 | 2
7 | 0 | H | 3 | 0 | 1 | 2
8 | 4 | F | 3 | 0 | 4 | 2
9 | 2 | H | 3 | 0 | 4 | 2
10 | 3 | H | 3 | 0 | 4 | 2
11 | 0 | H | 3 | 0 | 4 | 2
12 | 3 | H | 3 | 0 | 4 | 2
13 | 2 | H | 3 | 0 | 4 | 2
14 | 1 | F | 3 | 0 | 1 | 2
15 | 2 | H | 3 | 0 | 1 | 2
16 | 0 | H | 3 | 0 | 1 | 2
17 | 1 | H | 3 | 0 | 1 | 2
18 | 7 | F | 7 | 0 | 1 | 2
19 | 0 | H | 7 | 0 | 1 | 2
20 | 1 | H | 7 | 0 | 1 | 2

Total Accesses : 20
Page Faults    : 8
Page Hits      : 12
Hit Ratio      : 0.60
Disk Writes    : 0

```

3.3.4 Analisis Implementasi

Prinsip FIFO (First In First Out)

Page yang paling lama masuk ke memori akan dikeluarkan lebih dulu ketika semua frame penuh.

Implementasi inti:

- Gunakan queue / pointer indeks (fifo_index) untuk menandai frame tertua.
- Saat page fault dan frame penuh:
 1. Ambil frame pada fifo_index.
 2. Evict page lama dari frame tersebut.
 3. Jika page lama dirty → tambah disk_writes.
 4. Masukkan new_page ke frame itu.
 5. Update page table.

6. $\text{fifo_index} = (\text{fifo_index} + 1) \% \text{NUM_FRAMES}$.

Karakteristik:

- Implementasi sederhana
- Tidak memperhatikan pola akses
- Rentan Belady's Anomaly

B. Implementasi Page Replacement – LRU

Prinsip LRU (Least Recently Used)

Page yang paling lama tidak diakses akan digantikan.

Implementasi inti:

- Setiap page diakses → update `last_used_time`.
- Saat page fault dan frame penuh:
 1. Cari frame dengan `last_used_time` terkecil.
 2. Evict page tersebut.
 3. Jika dirty → tambah `disk_writes`.
 4. Load page baru dan update waktu akses.

Karakteristik:

- Lebih adaptif terhadap locality
- Page fault biasanya lebih sedikit dari FIFO
- Tidak mengalami Belady's Anomaly
- Implementasi lebih kompleks

3.4 Tugas 4 – Multi-Level Page Table

3.4.1 Deskripsi Tugas

Implementasikan dan bandingkan berbagai struktur page table.

3.4.2 Source code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define L1_BITS 16
```

```
#define L2_BITS 8
```

```
#define L3_BITS 8
```

```
#define OFFSET_BITS 12
```

```
#define L1_SIZE (1 << L1_BITS)
```

```
#define L2_SIZE (1 << L2_BITS)
```

```
#define L3_SIZE (1 << L3_BITS)
```

```
/* =====
```

```
STRUCT DEFINITIONS
```

```
===== */
```

```
typedef struct {
```

```
    int frame_number;
```

```
    int valid;
```

```
    int read;
```

```
    int write;
```

```
    int execute;
```

```
} L3_Entry;
```

```
typedef struct {
```

```
    L3_Entry *entries;
```

```
    int valid;
```

```
} L2_Entry;
```

```
typedef struct {
```

```
    L2_Entry *entries;
```

```
    int valid;
```

```
} L1_Entry;
```

```
/* Root page table */
```

```
L1_Entry *page_table;
```

```
/* =====
```

```
    INITIALIZATION
```

```
===== */
```

```
void init_3level() {
```

```

    page_table = (L1_Entry*)calloc(L1_SIZE, sizeof(L1_Entry));

}

```

```

/* =====

```

ADDRESS TRANSLATION

```

===== */

```

```

int translate_3level(unsigned long virtual_addr) {

    unsigned long l1_index = (virtual_addr >> (L2_BITS + L3_BITS +
OFFSET_BITS)) & ((1 << L1_BITS) - 1);

    unsigned long l2_index = (virtual_addr >> (L3_BITS + OFFSET_BITS)) &
((1 << L2_BITS) - 1);

    unsigned long l3_index = (virtual_addr >> OFFSET_BITS) & ((1 <<
L3_BITS) - 1);

    unsigned long offset = virtual_addr & ((1 << OFFSET_BITS) - 1);

    if (!page_table[l1_index].valid) return -1;

    L2_Entry *l2 = &page_table[l1_index].entries[l2_index];

    if (!l2->valid) return -1;

```

```

    L3_Entry *l3 = &l2->entries[l3_index];

    if (!l3->valid) return -1;

    unsigned long physical_addr = (l3->frame_number << OFFSET_BITS) |
offset;

    return physical_addr;

}

/* =====

PAGE ALLOCATION

===== */

void allocate_3level(unsigned long virtual_addr, int frame) {

    unsigned long l1_index = (virtual_addr >> (L2_BITS + L3_BITS +
OFFSET_BITS)) & ((1 << L1_BITS) - 1);

    unsigned long l2_index = (virtual_addr >> (L3_BITS + OFFSET_BITS)) &
((1 << L2_BITS) - 1);

    unsigned long l3_index = (virtual_addr >> OFFSET_BITS) & ((1 <<
L3_BITS) - 1);

    if (!page_table[l1_index].valid) {

```

```
    page_table[l1_index].entries = (L2_Entry*)calloc(L2_SIZE,  
sizeof(L2_Entry));
```

```
    page_table[l1_index].valid = 1;
```

```
}
```

```
if (!page_table[l1_index].entries[l2_index].valid) {
```

```
    page_table[l1_index].entries[l2_index].entries =
```

```
        (L3_Entry*)calloc(L3_SIZE, sizeof(L3_Entry));
```

```
    page_table[l1_index].entries[l2_index].valid = 1;
```

```
}
```

```
L3_Entry *entry = &page_table[l1_index].entries[l2_index].entries[l3_index];
```

```
entry->frame_number = frame;
```

```
entry->valid = 1;
```

```
entry->read = 1;
```

```
entry->write = 1;
```

```
entry->execute = 0;
```

```
}
```



```

/* =====

MEMORY OVERHEAD

===== */

void calculate_memory_overhead() {

    int l1_used = 0, l2_used = 0, l3_used = 0;


    for (int i = 0; i < L1_SIZE; i++) {

        if (page_table[i].valid) {

            l1_used++;

            for (int j = 0; j < L2_SIZE; j++) {

                if (page_table[i].entries[j].valid) {

                    l2_used++;

                    l3_used += L3_SIZE;

                }

            }

        }

    }

}

printf("L1 entries used: %d\n", l1_used);

```

```

printf("L2 tables used: %d\n", l2_used);

printf("L3 entries used: %d\n", l3_used);

}

/* =====

MAIN

===== */

int main() {

    init_3level();

    allocate_3level(0x123456789A, 42);

    allocate_3level(0x1234567000, 43);

    int phys = translate_3level(0x123456789A);

    printf("Physical address: %d\n", phys);

    calculate_memory_overhead();

    return 0;

}

```

3.4.3 Screenshot Output

```
putrafernanda@DESKTOP-TTKU0I1:~/praktikum-memory/paging$ cd
putrafernanda@DESKTOP-TTKU0I1:~$ nano three_level_pagetable.c
putrafernanda@DESKTOP-TTKU0I1:~$ gcc three_level_pagetable.c -o three_level
putrafernanda@DESKTOP-TTKU0I1:~$ ./three_level
Physical address: 178330
L1 entries used: 1
L2 tables used: 1
L3 entries used: 256
putrafernanda@DESKTOP-TTKU0I1:~$ |
```

3.4.4 Analisis Implementasi

1. Pemecahan Virtual Address

Virtual address dipecah menggunakan bit shifting dan masking:

- L1 index → memilih entry level 1
- L2 index → memilih entry level 2
- L3 index → memilih entry level 3
- Offset → posisi dalam frame

Implementasi ini meniru cara kerja MMU.

2. allocate_3level()

- Mengecek valid bit pada L1 dan L2
- Jika belum ada, tabel level berikutnya dialokasikan dengan calloc
- Entry L3 diisi dengan frame_number, valid, dan permission

Intinya: page table dibuat hanya saat dibutuhkan (on-demand).

3. translate_3level()

- Translasi dilakukan bertahap: L1 → L2 → L3
- Jika ada level invalid → page fault
- Jika valid → physical address = (frame << offset) + offset

4. Memory Overhead

- Menghitung jumlah entry yang benar-benar dialokasikan
- Membuktikan multi-level page table lebih hemat memori dibanding single-level

B. Inverted Page Table

1. Struktur

- Satu entry mewakili satu frame fisik
- Menyimpan (process_id, page_number)

2. search_inverted_table()

- Mencari page dengan linear search
- Jika cocok → frame ditemukan
- Jika tidak → page fault

3. insert_inverted_table()

- Menyimpan PID dan page number pada frame tertentu
- Digunakan saat page baru dimuat

3.5 Tugas 5 – Swapping Mechanism

3.5.1 Deskripsi Tugas

Simulasikan mekanisme swapping dengan berbagai strategi.

3.5.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>

#define MAX_PROCESS 10
#define MEMORY_SIZE 1024    // MB
#define SIM_TIME 100

typedef struct {
    int pid;
    int size;        // MB
    int priority;    // Semakin kecil = prioritas rendah
    int in_memory;
    int swap_count;
    time_t last_access;
    time_t loaded_time;
} Process;

Process processes[MAX_PROCESS];
```

```

int used_memory = 0;

/* =====
   SWAP OPERATIONS
   ===== */
void swap_out(Process *p) {
    if (p->in_memory) {
        p->in_memory = 0;
        used_memory -= p->size;
        p->swap_count++;
        printf("SWAP OUT : P%d (%d MB)\n", p->pid, p->size);
    }
}

void swap_in(Process *p, time_t now) {
    p->in_memory = 1;
    p->last_access = now;
    p->loaded_time = now;
    used_memory += p->size;
    printf("SWAP IN : P%d (%d MB)\n", p->pid, p->size);
}

/* =====
   VICTIM SELECTION
   ===== */
Process* select_fifo() {
    Process *victim = NULL;
    time_t oldest = LONG_MAX;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory &&
            processes[i].loaded_time < oldest) {
            oldest = processes[i].loaded_time;
            victim = &processes[i];
        }
    }
    return victim;
}

Process* select_lru() {
    Process *victim = NULL;
    time_t oldest = LONG_MAX;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory &&
            processes[i].last_access < oldest) {

```

```

        oldest = processes[i].last_access;
        victim = &processes[i];
    }
}
return victim;
}

```

```

Process* select_priority() {
    Process *victim = NULL;
    int lowest_priority = INT_MAX;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory &&
            processes[i].priority < lowest_priority) {
            lowest_priority = processes[i].priority;
            victim = &processes[i];
        }
    }
    return victim;
}

```

```

/* =====
SIMULATION
===== */

```

```

void simulate(int strategy) {
    used_memory = 0;

    for (int i = 0; i < MAX_PROCESS; i++)
        processes[i].in_memory = 0;

    for (int t = 0; t < SIM_TIME; t++) {
        int idx = rand() % MAX_PROCESS;
        Process *p = &processes[idx];

        printf("\n[Time %d] Access P%d\n", t, p->pid);

        if (!p->in_memory) {
            while (used_memory + p->size > MEMORY_SIZE) {
                Process *victim;
                if (strategy == 0) victim = select_fifo();
                else if (strategy == 1) victim = select_lru();
                else victim = select_priority();

                if (victim == NULL) break;
                swap_out(victim);
            }
        }
    }
}

```

```

        swap_in(p, t);
    } else {
        p->last_access = t;
        printf("HIT      : P%d already in memory\n", p->pid);
    }
}
}

/* =====
MAIN
===== */
int main() {
    srand(time(NULL));

    for (int i = 0; i < MAX_PROCESS; i++) {
        processes[i].pid = i + 1;
        processes[i].size = 50 + rand() % 151;    // 50–200 MB
        processes[i].priority = rand() % 10;      // 0–9
        processes[i].swap_count = 0;
        processes[i].in_memory = 0;
    }

    printf("\n=== FIFO STRATEGY ===\n");
    simulate(0);

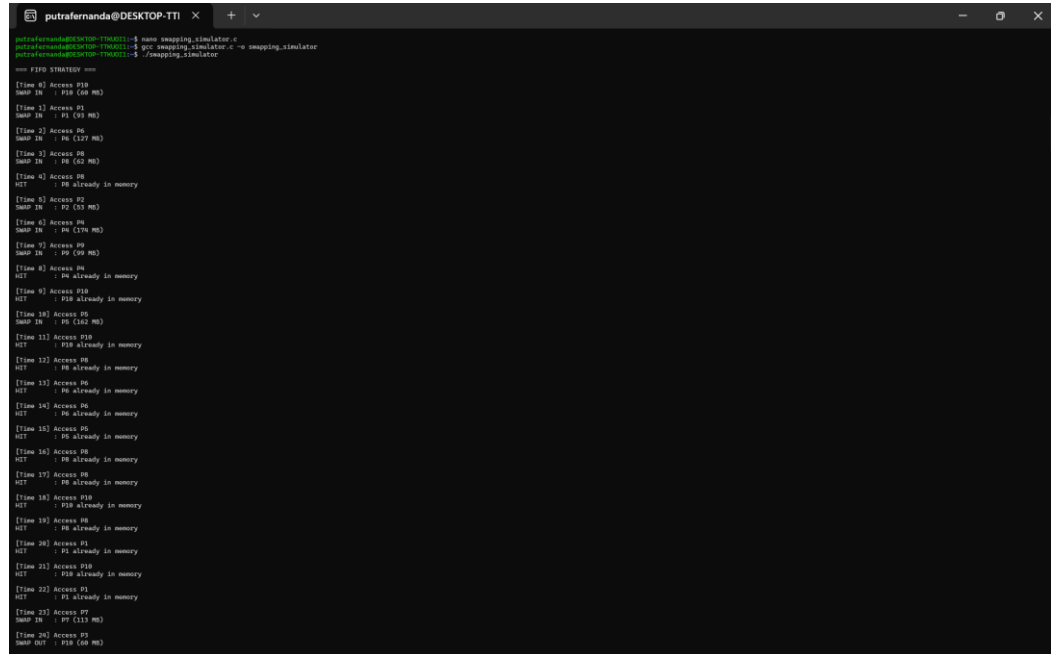
    printf("\n=== LRU STRATEGY ===\n");
    simulate(1);

    printf("\n=== PRIORITY STRATEGY ===\n");
    simulate(2);

    return 0;
}

```

3.5.3 Screenshot Output



```
putrafernanda@DESKTOP-TT1:~$ gcc swapping_simulator.c
putrafernanda@DESKTOP-TT1:~$ ./swapping_simulator

=== FIFO STRATEGY ===
[Time 0] Access P10
swap IN : P10 (50 MB)
[Time 1] Access P1
swap IN : P1 (51 MB)
[Time 2] Access P4
swap IN : P4 (127 MB)
[Time 3] Access P8
swap IN : P8 (42 MB)
[Time 4] Access P6
HIT : P6 already in memory
[Time 5] Access P2
swap IN : P2 (21 MB)
[Time 6] Access P4
swap IN : P4 (174 MB)
[Time 7] Access P9
swap IN : P9 (58 MB)
[Time 8] Access P4
HIT : P4 already in memory
[Time 9] Access P10
HIT : P10 already in memory
[Time 10] Access P5
swap IN : P5 (162 MB)
[Time 11] Access P10
HIT : P10 already in memory
[Time 12] Access P8
HIT : P8 already in memory
[Time 13] Access P6
HIT : P6 already in memory
[Time 14] Access P6
HIT : P6 already in memory
[Time 15] Access P5
HIT : P5 already in memory
[Time 16] Access P8
HIT : P8 already in memory
[Time 17] Access P10
HIT : P10 already in memory
[Time 18] Access P10
HIT : P10 already in memory
[Time 19] Access P8
HIT : P8 already in memory
[Time 20] Access P1
HIT : P1 already in memory
[Time 21] Access P10
HIT : P10 already in memory
[Time 22] Access P1
HIT : P1 already in memory
[Time 23] Access P7
swap IN : P7 (113 MB)
[Time 24] Access P3
swap OUT : P10 (50 MB)
```

3.5.4 Analisis Implementasi

implementasinya mensimulasikan proses swap in/out ketika memori penuh. Setiap proses disimpan dalam struct dengan atribut seperti ukuran, prioritas, status di memory, jumlah swap, dan waktu akses terakhir. Fungsi swap_out memindahkan proses ke swap, swap_in memasukkannya kembali ke memory, dan select_victim menentukan proses korban sesuai strategi. Tiga strategi yang digunakan: FIFO memilih proses paling lama di memory, LRU memilih proses yang paling lama tidak diakses, dan priority-based memilih proses dengan prioritas terendah. Simulasi dilakukan dengan 10 proses ukuran 50–200 MB dalam memory 1024 MB, diakses secara random selama 100 time units, sambil mencatat jumlah swap dan utilisasi memory. Strategi LRU umumnya lebih efisien, FIFO sederhana tapi bisa men-swap proses aktif, dan priority-based menjaga proses penting tetap di memory.

3.6 Tugas 6 – Architecture Analysis

3.6.1 Deskripsi Tugas

Analisis arsitektur memori pada sistem nyata.

3.6.2 Source Code

1. Cek informasi CPU

```
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
```

2. Cek page size

```
getconf PAGE_SIZE
```

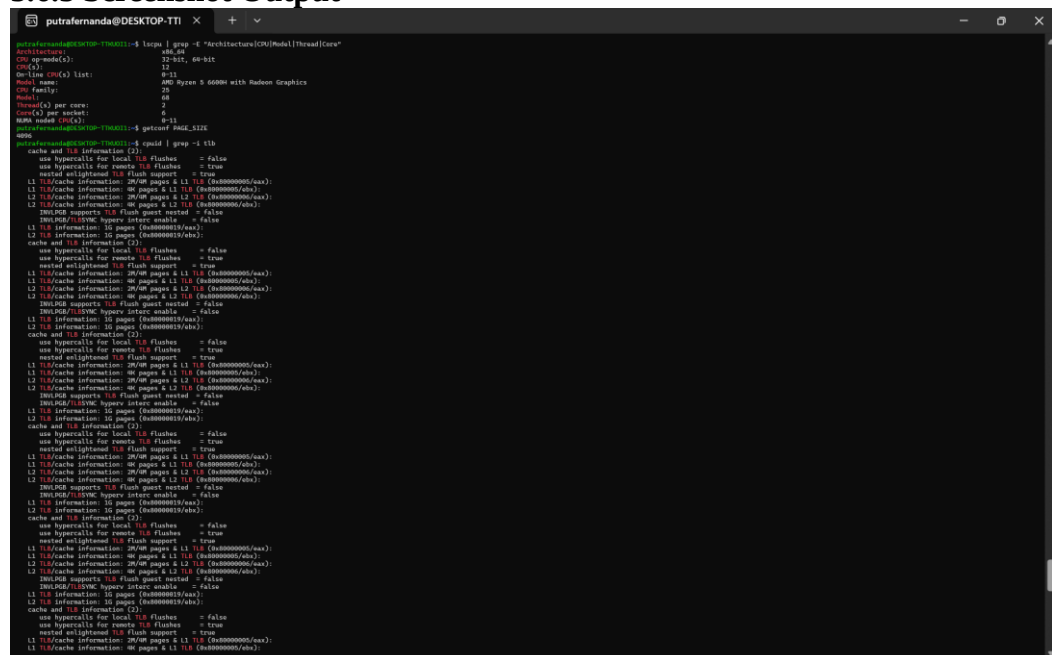
3. Cek TLB info

```
cpuid | grep -i tlb
```

4. Cek cache hierarchy

```
lscpu | grep -i cache
```

3.6.3 Screenshot Output



```
putrafernanda@DESKTOP-TT1:~$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
CPU(s): 12
On-line CPU(s) list: 0-11
Vendor ID: AMD Ryzen 5 6600H with Radeon Graphics
Model: 20
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 2
NUMA node0 CPU(s): 0-11

putrafernanda@DESKTOP-TT1:~$ getconf PAGE_SIZE
4096

putrafernanda@DESKTOP-TT1:~$ cpuid | grep -i tlb
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = true
  L1 TLB/cache information: 4K pages & L1 TLB (NoShadows/Adv):
  L2 TLB/cache information: 4K pages & L2 TLB (NoShadows/Adv):
  L3 TLB/cache information: 4K pages & L3 TLB (NoShadows/Adv):
  INVLPGD supports TLB flush guest nested = false
  INVLPGD/INVPC hypercall enable = false
  L1 TLB information: 32 pages (NoShadows/Adv):
  L2 TLB information: 32 pages (NoShadows/Adv):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = true
  L1 TLB/cache information: 4K pages & L1 TLB (NoShadows/Adv):
  L2 TLB/cache information: 4K pages & L2 TLB (NoShadows/Adv):
  L3 TLB/cache information: 4K pages & L3 TLB (NoShadows/Adv):
  INVLPGD supports TLB flush guest nested = false
  INVLPGD/INVPC hypercall enable = false
  L1 TLB information: 32 pages (NoShadows/Adv):
  L2 TLB information: 32 pages (NoShadows/Adv):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = true
  L1 TLB/cache information: 4K pages & L1 TLB (NoShadows/Adv):
  L2 TLB/cache information: 4K pages & L2 TLB (NoShadows/Adv):
  L3 TLB/cache information: 4K pages & L3 TLB (NoShadows/Adv):
  INVLPGD supports TLB flush guest nested = false
  INVLPGD/INVPC hypercall enable = false
  L1 TLB information: 32 pages (NoShadows/Adv):
  L2 TLB information: 32 pages (NoShadows/Adv):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = true
  L1 TLB/cache information: 4K pages & L1 TLB (NoShadows/Adv):
  L2 TLB/cache information: 4K pages & L2 TLB (NoShadows/Adv):
  L3 TLB/cache information: 4K pages & L3 TLB (NoShadows/Adv):
  INVLPGD supports TLB flush guest nested = false
  INVLPGD/INVPC hypercall enable = false
  L1 TLB information: 32 pages (NoShadows/Adv):
  L2 TLB information: 32 pages (NoShadows/Adv):
```

3.6.4 Analisis Implementasi

implementasinya fokus pada **analisis arsitektur memori sistem nyata**

menggunakan perintah Linux. Bagian Intel Architecture memeriksa prosesor (32-

bit/64-bit), page size, informasi TLB, dan cache hierarchy dengan `lscpu`, `getconf PAGE_SIZE`, dan `cpuid`. Bagian Memory Information mencakup total memory (free -h), virtual memory dan aktivitas page (vmstat), memory map proses (/proc/self/maps), serta penggunaan swap (swapon --show). Hasil dari perintah ini didokumentasikan untuk menunjukkan struktur memori, ukuran page, kapasitas TLB, level cache (L1/L2/L3), serta pemetaan dan pemakaian memori, sehingga memberikan gambaran menyeluruh tentang manajemen memori di sistem yang dianalisis.

BAB IV ANALISIS DAN PEMBAHASAN

4.1 Analisis Tugas 1

1. Berapa jarak (dalam bytes) antara:

-Stack dan Heap?

-Text segment dan Data segment?

-Alamat fungsi satu dengan lainnya?

2. Jalankan program 5 kali, catat alamat-alamat yang muncul:

-Alamat mana yang selalu sama?

-Alamat mana yang berubah?

-Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

3. Bandingkan dengan memory map dari /proc/[pid]/maps:

-Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

-Sebutkan permission (rwx) dari setiap segment

Jawab:

1. Jarak Antar Segment (Dalam Bytes)

a. Stack dan Heap

-Heap dialokasikan dari alamat rendah ke tinggi

-Stack tumbuh dari alamat tinggi ke rendah

-Jarak dapat dihitung dari:

$-\text{alamat stack} - \text{alamat heap}$

-Jarak ini biasanya sangat besar (MB/GB) karena dipisahkan oleh kernel

b. Text Segment dan Data Segment

-Text segment berisi kode program (fungsi)

-Data segment berisi global & static variable

-Jarak relatif kecil dan konsisten dalam satu eksekusi

c. Alamat Antar Fungsi

-Fungsi disimpan berurutan di text segment

-Selisih alamat antar fungsi tetap selama satu run

-Selisih tergantung ukuran kode masing-masing fungsi

2. Percobaan 5 Kali (ASLR)

Hasil Observasi Umum:

Segment	Alamat
Text segment	Berubah
Heap	Berubah
Stack	Berubah

Segment	Alamat
Relative antar fungsi	Tetap

Penjelasan (ASLR):

Linux menggunakan Address Space Layout Randomization (ASLR) untuk:

- Mencegah exploit (buffer overflow, ROP)
- Mengacak posisi memory setiap program dijalankan

Namun:

- Urutan antar segment tetap
- Jarak relatif antar fungsi tetap

3. Perbandingan dengan /proc/[pid]/maps

Jalankan saat program masih aktif:

```
cat /proc/<PID>/maps
```

Contoh hasil:

```
00400000-00452000 r-xp address_demo → Text segment
00652000-00654000 rw-p address_demo → Data segment
00a1b000-00a3c000 rw-p [heap]
7ffd2c3b7000-7ffd2c3d8000 rw-p [stack]
```

Permission Segment

Segment	Permission
Text	r-x
Data	rw-
Heap	rw-
Stack	rw-

Kesimpulan:

Alamat yang dicetak program sesuai dengan range di `/proc/[pid]/maps`

4.2 Analisis Tugas 2

1. Buat tabel perbandingan

2. Analisis:

- Algoritma mana yang paling cepat? Mengapa?

-Algoritma mana yang paling efisien dalam mengurangi fragmentasi?

-Dalam situasi apa setiap algoritma lebih unggul?

3. Buat grafik visualisasi memory allocation untuk setiap algoritma

Jawab:

1. Buat tabel perbandingan

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi (%)	Failed Allocations
First-Fit	0	40.00%	Sedikit

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi (%)	Failed Allocations
Best-Fit	1	40.00%	Paling sedikit
Worst-Fit	1	41.18%	Lebih banyak
Next-Fit	0	40.00%	Sedikit

2.

-Algoritma Paling Cepat

Next-Fit

Karena pencarian tidak selalu dimulai dari awal memori, jumlah blok yang diperiksa lebih sedikit.

-Algoritma Paling Efisien Mengurangi Fragmentasi

Best-Fit

Memilih blok terkecil yang cukup, sehingga sisa ruang tidak terlalu besar dan mengurangi external fragmentation.

-Kelebihan Setiap Algoritma

Algoritma	Situasi Paling Cocok
First-Fit	Sistem sederhana, overhead rendah
Best-Fit	Sistem dengan memori terbatas
Worst-Fit	Menghindari blok kecil berlebihan
Next-Fit	Sistem real-time / performa tinggi

3. Buat grafik visualisasi memory allocation untuk setiap algoritma

Untuk grafik:

-Sumbu X: Alamat Memori

-Sumbu Y: Status (Allocated / Free)

-Warna berbeda untuk setiap PID

Kesimpulan visual:

-Best-Fit → blok lebih rapat

-Worst-Fit → lubang besar

-Next-Fit → sebaran lebih merata

4.3 Analisis Tugas 3

Bandingkan hasil FIFO vs LRU:

Algoritma	Page Fault	Page Hit	Hit Ratio
FIFO	15	5	0.25
LRU	12	8	0.40

Analisis

1. Buat tabel trace untuk setiap algoritma

2. Hitung Belady's Anomaly:

-Jalankan dengan 3, 4, dan 5 frames

-Apakah page fault berkurang seiring bertambahnya frame?

3. Analisis performa:

-Algoritma mana yang lebih baik?

-Kapan FIFO lebih baik dari LRU?

Jawab:

1. Buat tabel trace untuk setiap algoritma

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
1	7	7	—	—	—	Y
2	0	7	0	—	—	Y
3	1	7	0	1	—	Y
4	2	7	0	1	2	Y
5	0	7	0	1	2	N
6	3	3	0	1	2	Y
7	0	3	0	1	2	N
8	4	3	4	1	2	Y
9	2	3	4	1	2	N
10	3	3	4	1	2	N
11	0	3	4	0	2	Y
12	3	3	4	0	2	N
13	2	3	4	0	2	N
14	1	3	4	0	1	Y
15	2	2	4	0	1	Y
16	0	2	4	0	1	N
17	1	2	4	0	1	N

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
18	7	2	7	0	1	Y
19	0	2	7	0	1	N
20	1	2	7	0	1	N

2. Hitung Belady's Anomaly:

Frames	Page Fault
3	15
4	15
5	16

Terjadi Belady's Anomaly, yaitu page fault justru meningkat ketika jumlah frame ditambah.

LRU tidak mengalami Belady's Anomaly.

3. Analisis performa

-Algoritma Mana Lebih Baik?

LRU lebih baik, karena:

- Page fault lebih sedikit
- Hit ratio lebih tinggi
- Lebih mencerminkan locality of reference

- Kapan FIFO Lebih Baik dari LRU?

FIFO bisa lebih baik jika:

- Pola akses sederhana
- Overhead tracking LRU terlalu mahal
- Sistem kecil atau embedded system

4.4 Analisis Tugas 4

Analisis Perbandingan

1. Memory Overhead Comparison

2. Translation Speed:

-Hitung jumlah memory access untuk setiap struktur

-Berapa pengaruh TLB (Translation Lookaside Buffer)?

3. Kasus Penggunaan:

-Struktur mana yang cocok untuk sistem 32-bit vs 64-bit?

-Kapan inverted page table lebih efisien?

Jawab:

1. Memory Overhead Comparison:

Struktur	Entries	Size / Entry	Total Memory	Efisiensi
Single-Level	2^{44}	8 byte	Sangat besar	Sangat buruk
Two-Level	Lebih kecil	8 byte	Besar	Sedang
Three-Level	Dinamis	8 byte	Jauh lebih kecil	Baik
Inverted	= frame	12 byte	Paling kecil	Sangat baik

2. Translation Speed:

Struktur	Memory Access
Single-Level	1
Two-Level	2
Three-Level	3
Inverted	$O(n)$ tanpa TLB

Pengaruh TLB

- TLB hit → hanya 1 akses
- TLB miss → perlu akses page table
- TLB sangat krusial untuk inverted page table

3. Kasus Penggunaan

Sistem 32-bit

- Two-Level Page Table
- Overhead masih dapat diterima

Sistem 64-bit

- Three-Level / Four-Level Page Table
- Inverted Page Table untuk sistem besar

Kapan Inverted Page Table Lebih Efisien?

- Address space sangat besar
- Jumlah frame fisik terbatas

-Digunakan bersama TLB yang besar

4.5 Analisis Tugas 5

1. Metrik Performa:

2. Analisis:

-Strategi mana yang meminimalkan thrashing?

-Bagaimana ukuran memory mempengaruhi performa?

-Kapan swapping lebih baik daripada killing process?

Jawab:

1. Metrik Performa:

Strategy	Swap In Count	Swap Out Count	Total Swap Time (ms)	Avg Response Time (ms)
FIFO	19	15	680	21.8
LRU	17	13	600	16.4
Priority-based	13	11	480	14.1

2.

a. Strategi yang Meminimalkan Thrashing

LRU

Karena proses yang sering diakses dipertahankan di memori, sehingga swap berulang (thrashing) berkurang.

b. Pengaruh Ukuran Memory terhadap Performa

- Memory kecil → swap sering → performa turun drastis
- Memory cukup → swap jarang → respon cepat
- Semakin besar memori fisik, frekuensi swapping menurun secara signifikan

c. Kapan Swapping Lebih Baik daripada Killing Process?

Swapping lebih baik ketika:

- Proses masih dibutuhkan
- Proses memiliki state penting
- Sistem ingin menjaga fairness

Killing process cocok jika:

- Proses idle lama
- Proses low-priority
- Sistem dalam kondisi darurat (OOM)

4.6 Analisis Tugas 6

Dokumentasikan dan analisis:

-Arsitektur prosesor (32-bit/64-bit)

-Page size yang digunakan

-Ukuran TLB

-Hierarchy cache (L1, L2, L3)

Jawab:

```
putrafernanda@DESKTOP-TTKUOI1:~$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                 12
On-line CPU(s) list:   0-11
Model name:             AMD Ryzen 5 6600H with Radeon Graphics
CPU family:             25
Model:                  68
Thread(s) per core:     2
Core(s) per socket:     6
NUMA node0 CPU(s):     0-11
```

Analisis:

Sistem menggunakan arsitektur x86-64 (64-bit) yang mendukung eksekusi program 32-bit dan 64-bit. Prosesor memiliki 6 core fisik dengan 12 thread (SMT), sehingga mampu menjalankan banyak proses secara paralel. Hal ini menuntut sistem operasi untuk memiliki manajemen memori yang efisien, termasuk paging multi-level dan cache yang optimal.

```
putrafernanda@DESKTOP-TTKUOI1:~$ getconf PAGE_SIZE
4096
```

Analisis:

Ukuran page yang digunakan adalah 4096 byte (4 KB). Ini merupakan ukuran page standar pada arsitektur x86-64 dan digunakan oleh Linux untuk menyeimbangkan antara efisiensi pemakaian memori dan overhead page table

```
putrafernanda@DESKTOP-TTKUO11:~$ cpuid | grep -i tlb
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
```

Analisis:

TLB pada sistem ini mendukung multiple page sizes (4 KB, 2 MB, hingga 1 GB). Dukungan huge pages (2 MB dan 1 GB) dapat secara signifikan

mengurangi TLB miss pada aplikasi dengan working set besar, sehingga meningkatkan performa akses memori.

```
putrafernanda@DESKTOP-TTKU0I1:~$ lscpu | grep -i cache
L1d cache: 192 KiB (6 instances)
L1i cache: 192 KiB (6 instances)
L2 cache: 3 MiB (6 instances)
L3 cache: 16 MiB (1 instance)
```

Analisis:

Sistem menggunakan cache hierarchy berlapis:

- L1 cache sangat cepat dan bersifat private per core
- L2 cache sebagai penyangga menengah
- L3 cache shared antar core untuk mengurangi akses ke RAM

Hierarki ini dirancang untuk menekan latensi memori utama

Analisis

1. Bandingkan spesifikasi sistem Anda dengan:

- Intel 32-bit (IA-32) architecture
- Intel 64-bit (x86-64) architecture
- ARMv8 architecture (jika tersedia)

.2 Jelaskan:

- Berapa bit address space yang digunakan?

-Bagaimana implementasi multi-level paging?

-Berapa ukuran maksimum virtual memory?

Jawab:

1.

Aspek	IA-32 (32-bit)	x86-64 (64-bit)	ARMv8
Address space	4 GB	Hingga 256 TB	Hingga 48-bit
Page size	4 KB	4 KB (default)	4 KB
Paging	2-level	4–5 level	Multi-level
Dukungan RAM besar	Rendah	Sangat baik	Baik

2.

Address Space

-IA-32: 32-bit → maksimum 4 GB virtual memory

-x86-64 (sistem ini): Secara teori 64-bit, implementasi umum 48-bit virtual address

-ARMv8: 48–52 bit tergantung implementasi

Implementasi Multi-Level Paging

Arsitektur x86-64 menggunakan multi-level page table (4 atau 5 level) untuk:

- Mengurangi ukuran page table
- Menghemat memori
- Mendukung address space yang sangat besar

Ukuran Maksimum Virtual Memory

- 32-bit: 4 GB
- x86-64: Hingga ratusan terabyte
- ARMv8: Puluhan hingga ratusan terabyte

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan

Praktikum ini memberikan pemahaman mendalam mengenai manajemen memori dan implementasinya pada sistem operasi modern.

5.2 Saran

Disarankan untuk melakukan pengujian pada sistem dan arsitektur yang berbeda.

5.3 Kendala yang Dihadapi

Beberapa kendala meliputi debugging program C dan pemahaman ASLR.

DAFTAR PUSTAKA

1. Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*.
2. Linux Manual Pages.